TASK 1

1. Launch an EC2 Instance (Apache Web Server)

Steps:

1. Go to **EC2 Dashboard** → **Launch Instance**.
2. **Name**: Apache-Server
3. **AMI**: Choose **Ubuntu Server 22.04 LTS**
4. **Instance type**: t2.micro (Free tier eligible)
5. **Key pair**: Select existing or create a new key.
6. **Network settings**:
   - Create or select a Security Group:
     - Allow **SSH** (port 22)
     - Allow **HTTP** (port 80) – for Apache
7. **Launch the instance**

Install Apache:

bash

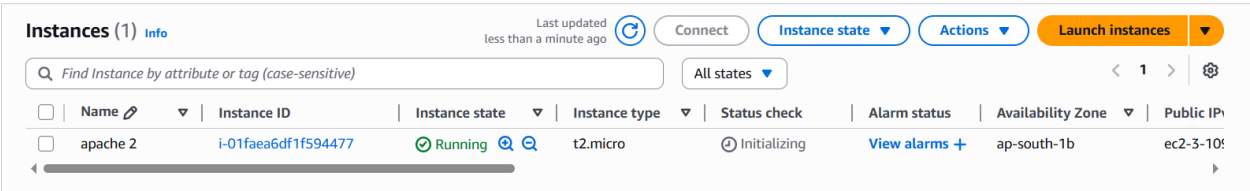
CopyEdit

sudo apt update

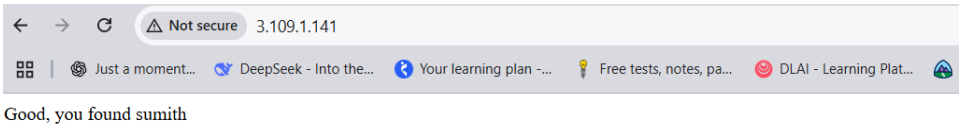
sudo apt install apache2 -y

Validate:

- Visit http://<EC2-Public-IP> → You should see **Apache2 Ubuntu Default Page**.



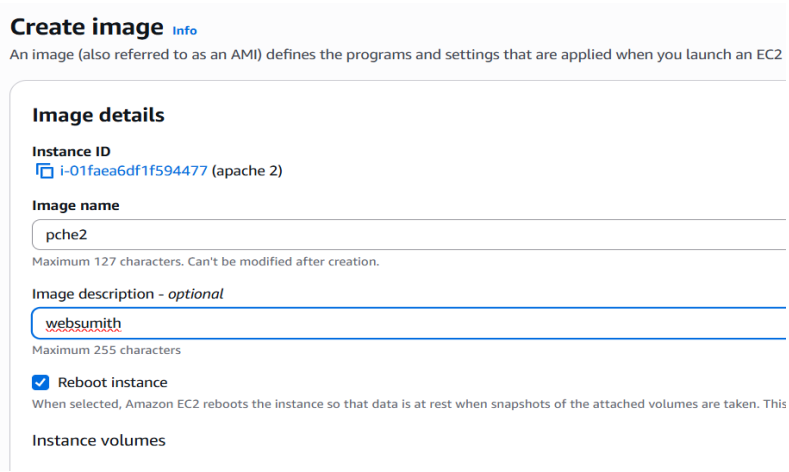

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable apache2
sudo systemctl restart  apache2
sudo systemctl status  apache2
cat > /var/www/html/index.html
cat > /var/www/html/index.html/
cat > var/www/html/index.html/
echo "Hello from Apache" | sudo tee /var/www/html/index.html
echo "Good, you found sumith" | sudo tee /var/www/html/index.html
```



2. Create a Custom AMI (My_Apache_AMI)

Steps:

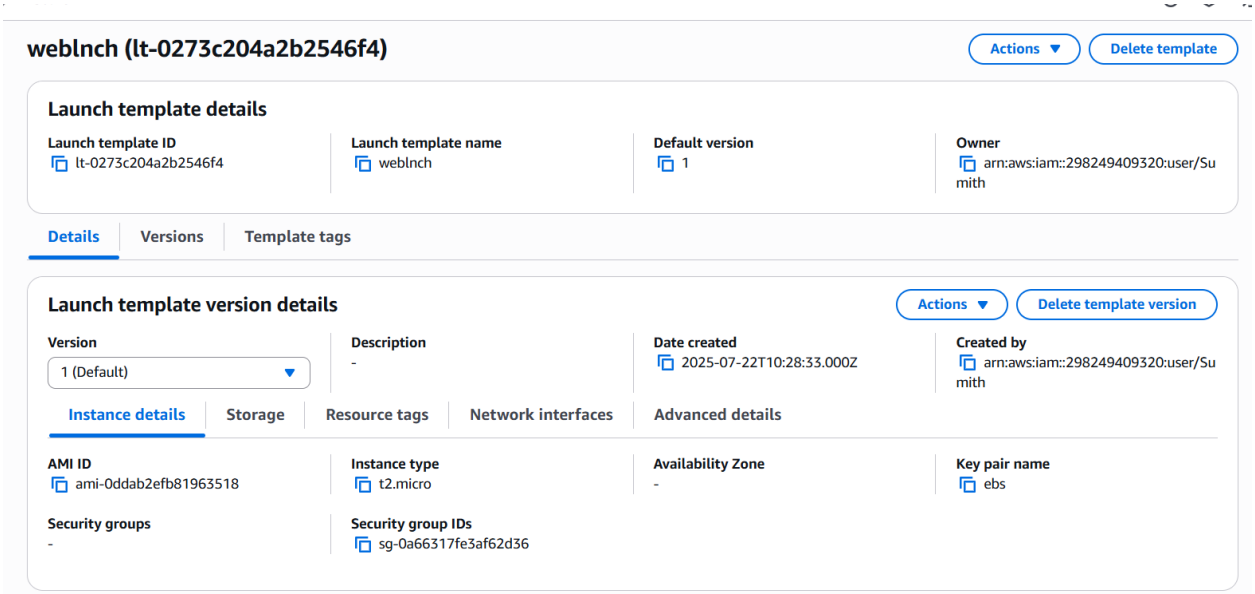
1. **Stop the EC2 Instance**
2. Go to **Instances** → Select the instance → **Actions** → Image → **Create Image**
3. Name it: My_Apache_AMI
4. Click **Create Image**



3. Create a Launch Template

Steps:

1. Go to **EC2** → **Launch Templates** → **Create Launch Template**
2. Name: ApacheTemplate
3. **AMI**: Select My_Apache_AMI
4. **Instance type**: t2.micro
5. **Key pair**: Choose existing
6. **Security Group**: Select one that allows HTTP (port 80)
7. Create template.



4. Create an Auto Scaling Group (ASG)

Steps:

1. Go to **Auto Scaling Groups** → **Create Auto Scaling Group**
2. Name: Apache-ASG
3. Select your **Launch Template**
4. **VPC and Subnets**: Select at least 2 subnets in different AZs
5. **Attach to Load Balancer?** → Skip for now.
6. **Desired capacity**: 1

   Minimum capacity: 1

   Maximum capacity: 3
7. **Health Check Type**: EC2

   Health Check Grace Period: 60 seconds
8. **Scaling Policies** → Create a policy:
   - **Target tracking**:
     - Metric: **Average CPU Utilization**
     - Target value: 60%
   - Or create a **Step Scaling Policy**:
     - Scale out: +1 instance if CPU > 60% for 2 mins

Name

Auto Scaling group name
Enter a name to identify the group.

sumithgs

Must be unique to this account in the current Region and no more than 255 characters.

Launch template Info

For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling grou configurations is not recommended but still available via the CLI and API until December 31, 2023.

Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

weblnch

Create a launch template

for getting started quickly.

VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.

vpc-0bca86d5a47543542
172.31.0.0/16 Default

Create a VPC

Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

- aps1-az1 (ap-south-1a) | subnet-0ebba556236d026fa
  172.31.32.0/20 Default
- aps1-az2 (ap-south-1c) | subnet-079d20354cba10c5b
  172.31.16.0/20 Default
- aps1-az3 (ap-south-1b) | subnet-05ce430bac8648065
  172.31.0.0/20 Default

Create a subnet

Group size Info

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to

Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for

Units (number of instances)

Desired capacity
Specify your group size.

1

Scaling Info

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits
Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity	Max desired capacity
1	3
Equal or less than desired capacity	Equal or greater than desired capacity

Auto Scaling group: sumithgs

Filter activity history

1

Status	Description	Cause	Start time	End time
Successful	Launching a new EC2 instance: i-032eb59059fda7f80	At 2025-07-22T10:47:29Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 1.	2025 July 22, 04:17:30 PM +05:30	2025 July 22, 04:17:35 PM +05:30
	Launching a new EC2 instance. Status Reason: Your requested instance			

Test Auto Scaling

A. Simulate CPU Load (to scale out):

1. SSH into the instance created by ASG:

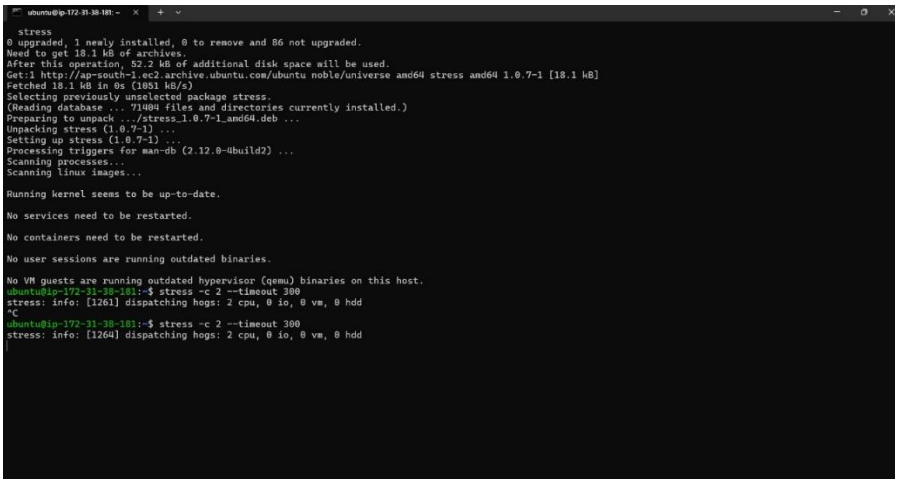
sudo apt update

sudo apt install stress -y

2. Run:

stress -c 2

This will spike CPU usage. After 2 minutes, a new instance should be added.



Status	Description	Cause
Not yet in service	Launching a new EC2 instance: i-0c70b1e829eee767a	At 2025-07-22T12:06:27Z a monitor alarm TargetTracking-myasg-AlarmHigh-2b9dc213-3aad-4b01-baa8-30819e45d84a in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 1 to 2. At 2025-07-22T12:06:32Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 2.
Waiting for instance warmup	Launching a new EC2 instance: i-0493b8924ecb106cf	At 2025-07-22T11:35:43Z a monitor alarm TargetTracking-grp-AlarmHigh-8abd53ad-d489-4cd1-aea6-65e8aa3a3ba9 in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 1 to 2. At 2025-07-22T11:35:52Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 2.

Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max	Availability Zones
grp	weblnch | Version Default	3	-	3	1	3	2 Availability Zones

i-0493b8924ecb106cf	Running	t2.micro	Initializing	View alarms +	ap-south-1b	ec2-13-2(
i-05e0407721369d6d0	Running	t2.micro	2/2 checks passed	View alarms +	ap-south-1a	ec2-13-2:

Successful	Launching a new EC2 instance: i-0493b8924ecb106cf	At 2025-07-22T11:35:43Z a monitor alarm TargetTracking-grp-AlarmHigh-8abd53ad-d489-4cd1-aea6-65e8aa3a3ba9 in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 1 to 2. At 2025-07-22T11:35:52Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 2.	2025 July 22, 05:05:53 PM +05:30	2025 July 22, 05:07:25 PM +05:30

TASK2

1. Scale Based on HTTP Requests

Goal: Auto-scale instances based on number of HTTP requests using an Application Load Balancer (ALB).

📌 **Theory:**

- **ALB** monitors traffic and routes HTTP(S) requests to healthy EC2 instances.
- **Target Group metrics** (like RequestCountPerTarget) are published to **CloudWatch**.
- **ASG Policy** can scale out/in based on these metrics.

⚙️ **Step-by-Step Setup:**

✅ **Step 1: Create/Prepare Your EC2 Setup**

- Launch an EC2 instance (Ubuntu preferred).
- Install and configure a web server (e.g., Apache or Nginx).

bash

CopyEdit

```
sudo apt update
sudo apt install apache2 -y
echo "Welcome to $(hostname)" | sudo tee /var/www/html/index.html
```

- Test by accessing http://<EC2-Public-IP> in the browser.

✅ **Step 2: Create a Custom AMI**

- Once your instance is ready and serving traffic:
  - Stop the instance.
  - Create an **AMI** (e.g., My_Apache_AMI).
  - This image will be reused by ASG to launch similar instances.

✅ **Step 3: Create an Application Load Balancer (ALB)**

1. Go to EC2 → Load Balancers → Create Load Balancer → **Application Load Balancer**.
2. Configure:
   - Scheme: **Internet-facing**
   - Listener: HTTP (port 80)
   - AZs/Subnets: Choose at least 2
3. Create a **Target Group**:
   - Target type: Instance
   - Protocol: HTTP, Port 80

    - Health check: Path = /
4. Skip instance registration for now (ASG will attach them later).
5. Finalize ALB creation.

✅ **Step 4: Create a Launch Template**

- EC2 → Launch Templates → Create new:
    - AMI: My_Apache_AMI
    - Instance type: t2.micro
    - Security Group: Allow **HTTP (80)** and **SSH (22)**
    - User Data (optional): Add script to auto-start web server on boot.

✅ **Step 5: Create Auto Scaling Group (ASG)**

- Go to EC2 → Auto Scaling Groups → Create:
    - Select your **Launch Template**
    - Choose **2 subnets (different AZs)** for high availability
    - Attach **Target Group** created above
    - Set **Desired = 1**, **Min = 1**, **Max = 3**

🔁 **Step 6: Add Scaling Policy Based on HTTP Requests**

📊 **Metric Used:**

- Request Count Per Target (per minute) from ALB Target Group

📌 **How It Works:**

- This metric tells how many requests each instance is receiving.
- Example logic:
    - If each instance receives **>1000 requests/minute**, add an instance.
    - If it drops below **200**, scale in.

⚙️ **Policy Setup:**

1. In your ASG → Go to **Automatic Scaling** → Add **Target Tracking Policy**.
2. Choose:
    - **ALB Request Count Per Target**
    - Target value = 1000 (you can test with smaller values)
    - Cooldown: 60 seconds
3. Save policy.

▯ Step 7: Generate Load to Trigger Scaling

Use Apache Benchmark or any load generator:

bash

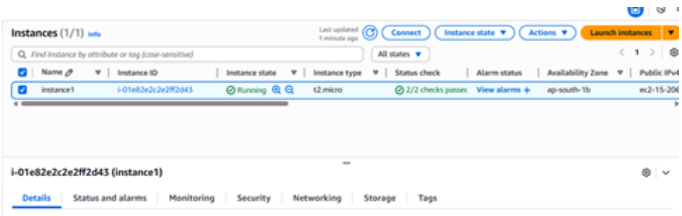
CopyEdit

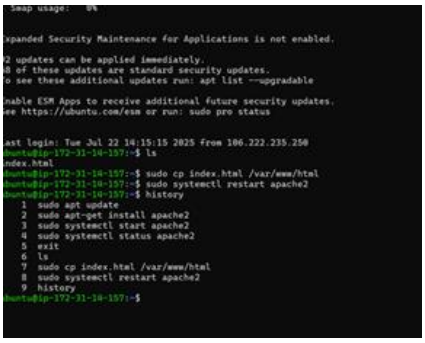
```
ab -n 10000 -c 100 http://<ALB-DNS-Name>/
```

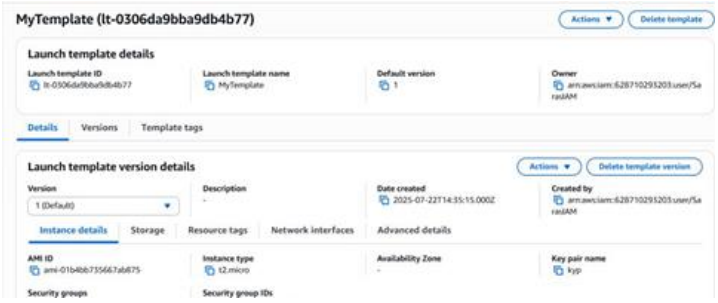
This sends 10,000 requests with 100 concurrent clients.

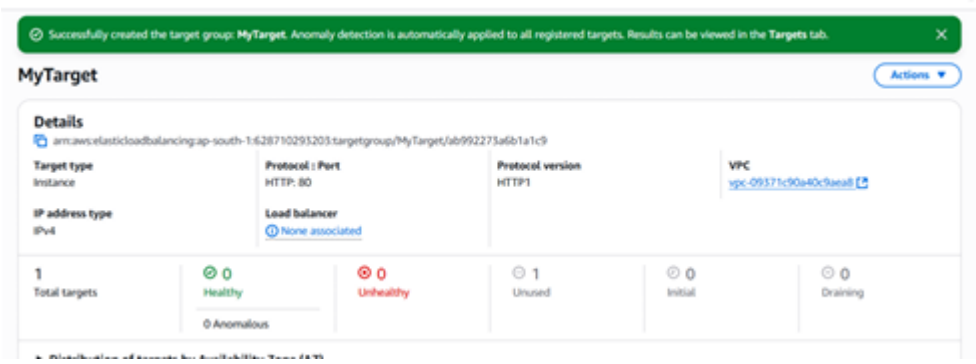
Watch in:

- **EC2 → ASG**: New instance should launch.
- **CloudWatch → Metrics → Target Group**

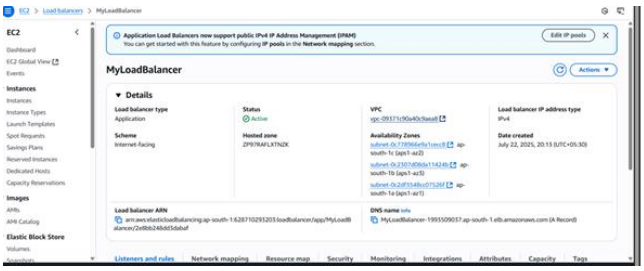


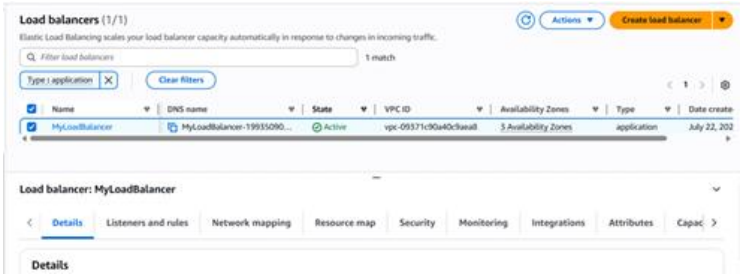


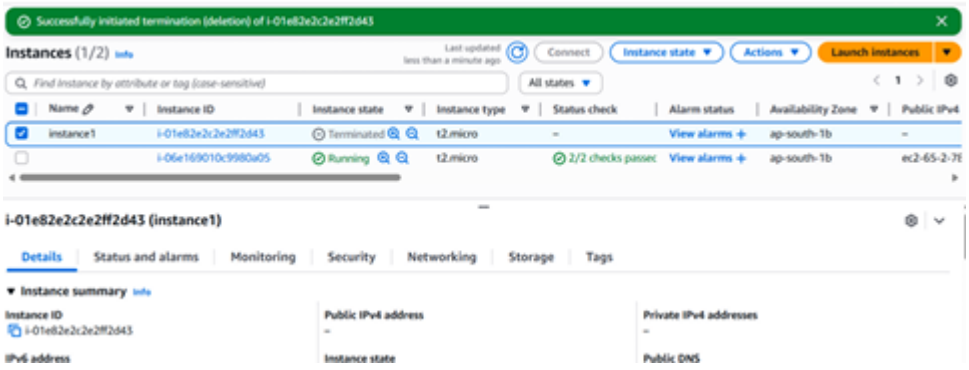


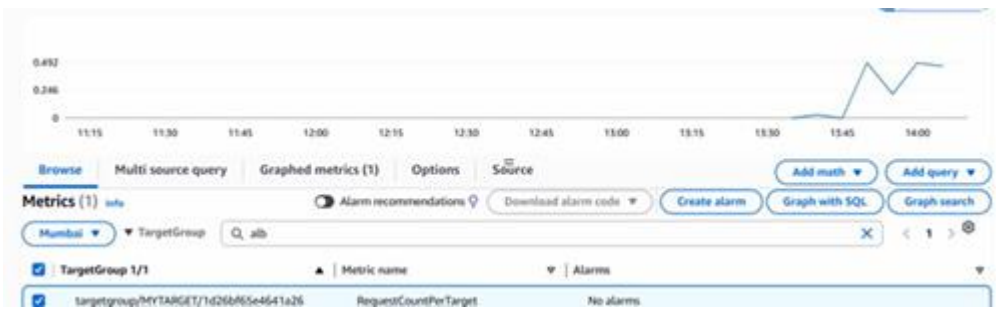


.









Task 3: Multi-AMI Testing:

🎯 Objective:

Create **two separate AMIs**, one with **Apache** and the other with **Nginx**, and compare how they behave when used in **Auto Scaling Groups (ASGs)** behind an **Application Load Balancer (ALB)**.

🔗 Why Do Multi-AMI Testing?

This allows you to:

Test Purpose	What to Observe
Boot Time	Which server (Apache/Nginx) starts faster?
Load Handling	Which AMI handles more HTTP requests efficiently?
Scaling Response	Does one scale more smoothly than the other?

Test Purpose	What to Observe
Health Check Compatibility	Any server failing ALB health checks?
Latency	Which server responds faster to clients?

🛠 Step-by-Step Setup

✅ Step 1: Create Two EC2 Instances

1. **Apache AMI Instance:**

bash

CopyEdit

```
sudo apt update
sudo apt install apache2 -y
echo "Apache - $(hostname)" | sudo tee /var/www/html/index.html
sudo systemctl enable apache2
```

2. **Nginx AMI Instance:**

bash

CopyEdit

```
sudo apt update
sudo apt install nginx -y
echo "Nginx - $(hostname)" | sudo tee /var/www/html/index.html
sudo systemctl enable nginx
```

✅ Step 2: Create Custom AMIs

- Stop each instance and create:
  - My_Apache_AMI
  - My_Nginx_AMI

✅ Step 3: Create Launch Templates

- Launch Template 1 → Uses My_Apache_AMI
- Launch Template 2 → Uses My_Nginx_AMI

Ensure both use same:

- Instance type (t2.micro)
- Security Group (allow ports 80 and 22)
- User data (if needed)

✅ Step 4: Create Two Auto Scaling Groups (ASG)

ame	Template	Group
pache	template	ache
ginx	emplate	nx

Each Target Group uses:

- HTTP
- Health Check Path = /

✅ Step 5: Create a Shared Application Load Balancer

- ALB with:
  - **Listener on HTTP (port 80)**
  - **Two Target Groups**:
    - TG-Apache with ASG-Apache
    - TG-Nginx with ASG-Nginx
- Set up **listener rules**:
  - For testing: direct /apache path to Apache TG, /nginx to Nginx TG

✅ Step 6: Generate Load Separately on Each

Use Apache Benchmark or any tool:

bash

CopyEdit

```
# For Apache
ab -n 1000 -c 50 http://<ALB-DNS>/apache/

# For Nginx
ab -n 1000 -c 50 http://<ALB-DNS>/nginx/
```

📊 **Metrics to Compare**

Metric	CloudWatch Source	What to Look For
Boot Time	EC2 → Launch logs	Nginx usually boots faster
CPU Utilization	ASG Metrics	Which consumes more CPU under load?
Request Latency	ALB Metrics → TargetResponseTime	Lower is better
Health Check Failures	Target Group	Which fails more often?
Auto Scaling Events	ASG Activity History	Who scales faster and better?

□ Conclusion:

You can now **compare the two stacks** like this:

Criteria	Apache	Nginx
Boot Time	Slower	Faster
Memory Usage	Higher	Lower
Request Handling	Thread-based	Event-driven (better for high concurrency)
Health Check Speed	Slightly Slower	Faster
Ideal For	Dynamic pages	Static pages / Reverse proxy

Troubleshooting Tasks

✅ **Simulate Failed Health Checks**

Goal: Test how ASG handles unhealthy instances.

📌 **Theory:**

- ASG checks instance health using **ELB health checks** or EC2 status checks.
- If health check fails (e.g., HTTP 5xx or connection refused), instance is marked **unhealthy**.
- ASG **terminates and replaces** it with a new one.

⚙️ **Steps:**

1. SSH into EC2 instance.
2. Run: sudo systemctl stop apache2
3. Wait 2–3 minutes.

4. ALB health check fails → ASG terminates the instance → New instance launched.

Tip: You can check ALB → Target Group → Targets tab to see status changes.

📊 4. Analyze CloudWatch Metrics During Scaling

📌 **Metrics to Watch:**

- CPUUtilization (from EC2/ASG)
- RequestCountPerTarget (from ALB Target Group)
- GroupInServiceInstances / GroupDesiredCapacity (ASG metrics)
- UnHealthyHostCount (Target Group)

🔍 **Interpretation:**

- **Spike in HTTP requests** → RequestCountPerTarget increases.
- If scaling policy is set (e.g., RequestCountPerTarget > 1000) → ASG launches more instances.
- After scaling, CPUUtilization may **drop** as load is distributed.
- During instance termination (in health check failure), GroupInServiceInstances drops temporarily.

▯ Summary Table

Task	Metric / Trigger	Action	Expected Outcome
Scale by HTTP Requests	RequestCountPerTarge	Scale out/in	More instances under load
Multi-AMI Testing	Load & latency	Compare Apache vs Nginx	Nginx usually faster
Simulate Health Failure	Manual service stop	Replace instance	ASG launches new one
CloudWatch Monitoring	PU, RequestCount, Hea	Analyze scaling trend	Track behavior visually